

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 3

Виконав:

студент групи кн-25-1

Рафієв Б. С.

Перевірив:

доцент кафедри КІЕ

Сидоренко В.М.

Кременчук 2026

Тема: Алгоритми сортування та їх складність. Порівняння алгоритмів сортування

Мета: опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

ХІД РОБОТИ

1. Бульбашкове сортування (Bubble Sort)

Алгоритм послідовно порівнює сусідні елементи та міняє їх місцями, якщо вони стоять не по порядку.

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        swapped = False  
        for j in range(0, n - i - 1):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
                swapped = True  
        if not swapped:  
            break  
    return arr
```

Складність та порівняння:

- **Найгірший випадок:** $O(n^2)$ (масив перевернутий).
- **Найкращий випадок:** $O(n^2)$ (масив уже відсортований, спрацьовує перевірка swapped).
- **Проти Insertion Sort:** Обидва мають $O(n^2)$, але Insertion Sort на практиці швидший, бо робить менше записів у пам'ять (зсуває елементи замість потрібного обміну swap).
- **Проти Merge Sort:** Бульбашка програє, бо виконує n^2 операцій, тоді як злиття – лише $n \log n$. При $n = 10000$ це 100 млн операцій проти 140 тисяч.

2. Складність Merge Sort за теоремою рекурсії

Рекурентне рівняння:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

За Основною теоремою:

- $a = 2, b = 2 \Rightarrow n^{\log_b a} = n^{\log_2 2} = n^1$
- Оскільки $f(n) = \Theta(n)$, це **2-й випадок** теореми.

Результат: $T(n) = \Theta(n \log n)$ у всіх випадках.

3. Швидке сортування (Quick Sort)

Обирається опорний елемент (pivot), масив ділиться на "менші за pivot" та "більші за pivot", які сортуються рекурсивно.

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[len(arr) // 2]  
    left = [x for x in arr if x < pivot]  
    middle = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]  
    return quick_sort(left) + middle + quick_sort(right)
```

Складність за теоремою рекурсії:

- **Найкращий випадок (ідеальний поділ навпіл):** Рівняння $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$. Аналогічно Merge Sort (2-й випадок теореми) дає $\Theta(n \log n)$.
- **Найгірший випадок (масив відсортований, поділ як 0 та n-1):** Рівняння $T(n) = T(n - 1) + \Theta(n)$. Теорема рекурсії **незастосовна** (немає ділення на b). Через суму прогресії складність стає $\Theta(n^2)$.
- **Висновки:** завдяки цій лабораторній роботі я вивчив поняття алгоритму, його властивостей, способів описання та методів оцінки складності..

Висновки: завдяки цій лабораторній роботі я опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

Відповіді на контрольні запитання

1. Що таке асимптотична складність алгоритму сортування і чому вона важлива для порівняння алгоритмів?

Асимптотична складність – це математичний спосіб оцінити швидкість роботи алгоритму залежно від кількості вхідних елементів. Вона критично важлива для порівняння, оскільки дозволяє зрозуміти, чи зможе програма обробити мільйони записів за секунди, чи вона "зависне" на години. Без цієї оцінки неможливо об'єктивно обрати найкращий метод сортування для конкретної задачі.

2. Які алгоритми сортування мають квадратичну складність у найгіршому випадку? Поясніть, чому це може бути проблемою для великих обсягів даних.

Квадратичну складність $O(n^2)$ у найгіршому випадку мають алгоритми бульбашкового, вставлянням та вибором. Це велика проблема для великих даних, оскільки при збільшенні масиву в 10 разів час сортування зростає у 100 разів. Наприклад, якщо 1000 елементів сортуються 1 секунду, то мільйон елементів на тому ж обладнанні зажадає понад 11 днів безперервної роботи.

3. В чому полягає перевага сортування злиттям над сортуванням вставками для великих наборів даних?

Сортування злиттям значно перевершує сортування вставлянням на великих масивах завдяки своїй складності $O(n \log n)$. Поки сортування вставлянням порівнює кожен елемент майже з кожним, злиття ефективно розбиває задачу на дрібні частини. На великих обсягах даних різниця між цими підходами стає прірвою, де злиття працює майже лінійно, а вставляння стає непридатним для використання.

4. Які алгоритми сортування використовуються для сортування списків у стандартних бібліотеках мов програмування, таких як Python, Java або C++?

У стандартних бібліотеках популярних мов зазвичай використовують складні гібридні алгоритми. Python та Java використовують Timsort, який поєднує переваги сортування злиттям та вставлянням. У C++ часто застосовується Introsort, який починає зі швидкого сортування, але перемикається на пірамідальне (Heap Sort), якщо рекурсія стає занадто глибокою, що гарантує стабільну швидкість.

5. Яка різниця між алгоритмами сортування злиттям і швидким сортуванням? У яких випадках краще використовувати кожен з цих алгоритмів?

Головна різниця між злиттям (Merge Sort) та швидким сортуванням (Quick Sort) полягає у використанні пам'яті та стабільності. Злиття завжди гарантує швидкість $O(n \log n)$, але потребує додаткового місця для копіювання масиву. Швидке сортування сортує елементи прямо "на місці", що економить пам'ять, проте в рідкісних випадках воно може сповільнюватися. Злиття краще для дуже великих даних і зовнішніх носіїв, а швидке сортування – для роботи в оперативній пам'яті.

6. Які фактори слід враховувати при виборі алгоритму сортування для конкретної задачі?

При виборі алгоритму слід враховувати обсяг вхідних даних, обмеження по оперативній пам'яті та тип самих даних. Також важливо знати, чи є дані вже частково впорядкованими та чи критична стабільність сортування – тобто збереження початкового порядку елементів із однаковими значеннями. Для критичних систем обирають алгоритми з гарантованим часом виконання, щоб уникнути непередбачуваних затримок.